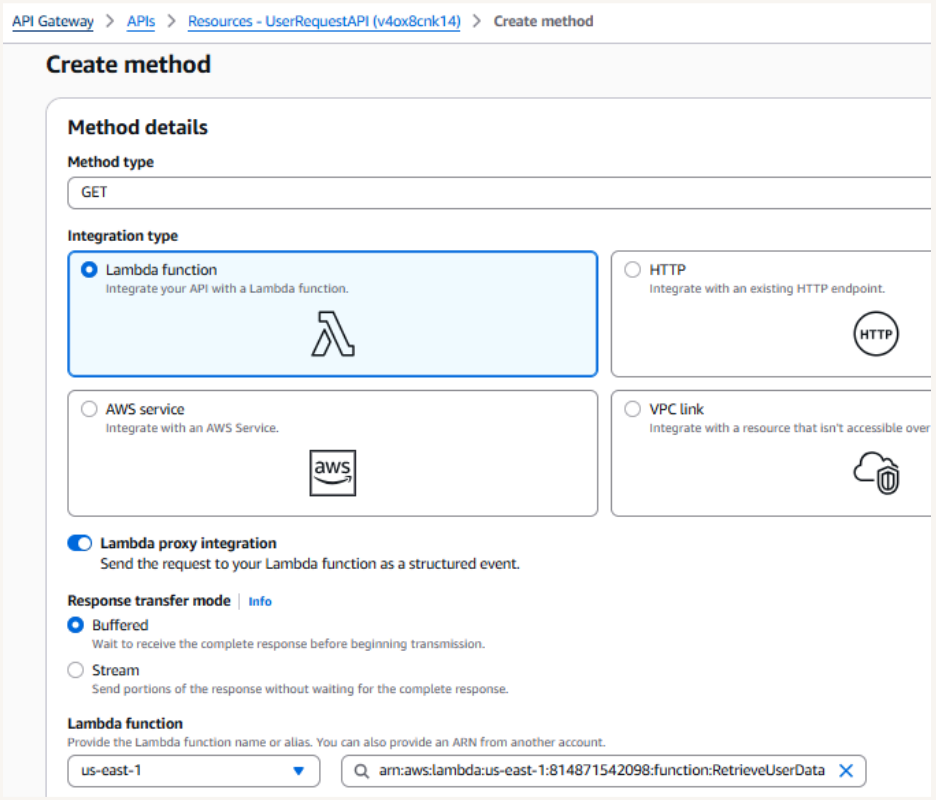
nextwork.org

APIs with Lambda + API Gateway

NI Nikhila Reddy



Introducing Today's Project!

In this project, I will demonstrate how to build and connect an API using AWS Lambda and Amazon API Gateway as part of a three-tier cloud architecture. I'm doing this project to learn how APIs allow communication between application layers and to better understand serverless workflows, API integrations, and backend cloud architecture.

Tools and concepts

Services I used in this project were Amazon API Gateway and AWS Lambda. Key concepts I learnt include Lambda functions, REST APIs, API resources and methods, Lambda proxy integration, API stages, invoke URLs, backend API workflows, and writing and publishing API documentation using API Gateway.

Project reflection

This project took me approximately 2 hours to complete, including the documentation and secret mission. The most challenging part was understanding how API Gateway resources, methods, and Lambda integrations work together as part of a backend workflow. It was most rewarding to successfully connect API Gateway with Lambda and understand how APIs manage requests between users and backend services.

placeholder

I chose to do this project today because I wanted to better understand how APIs work in serverless cloud architectures and how the logic tier of a three-tier application is built using AWS Lambda and API Gateway.

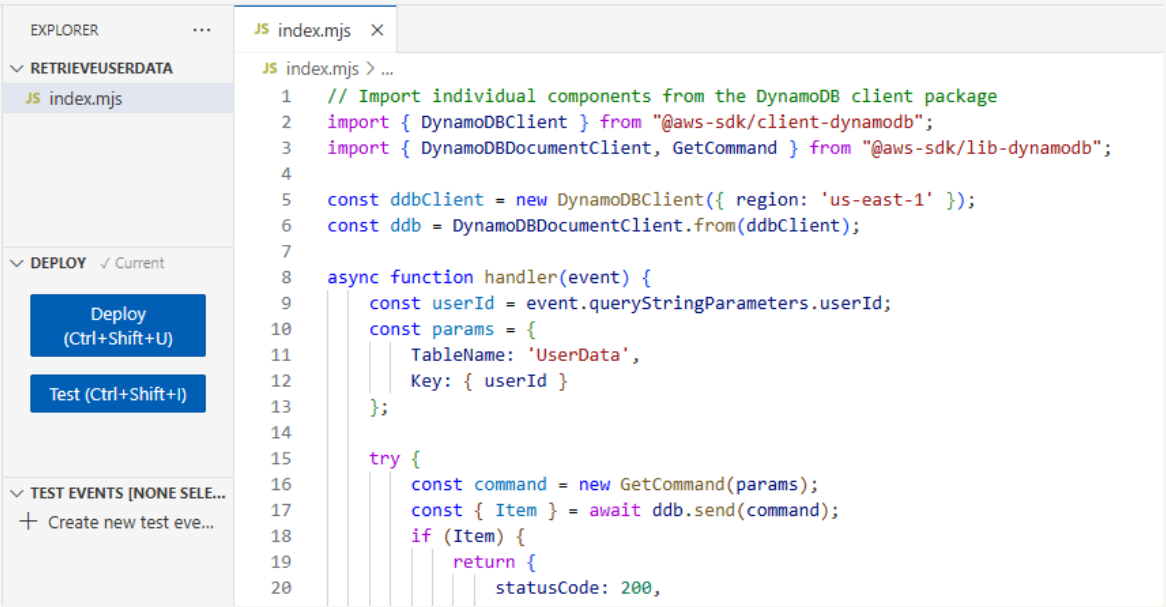
This project is also the second part of the three-tier architecture series, and since I had already completed the presentation tier and data tier projects, it helped me better understand how all three layers communicate together in a real-world web application. Next, I'll complete the final project where all three tiers are integrated into a complete three-tier web app. Something that would make learning with NextWork even better is adding a few updated screenshots or notes for newer AWS console UI changes, since some API Gateway workflows looked slightly different from the project documentation.

NI
Nikhila Reddy
NextWork Student

Lambda functions

AWS Lambda is a serverless computing service that lets developers run code without managing servers or infrastructure. It automatically handles scaling, execution, and resource management, allowing developers to focus only on writing application logic. In this project, I'm using AWS Lambda to process API requests, retrieve user data from a database, and act as the backend logic layer of a three-tier architecture.

The code I added to my Lambda function will receive a userId from an API request, query a DynamoDB table for matching user data, and return the results to the user in JSON format. The function also includes error handling to return appropriate responses if the user does not exist or if an error occurs during the request process.

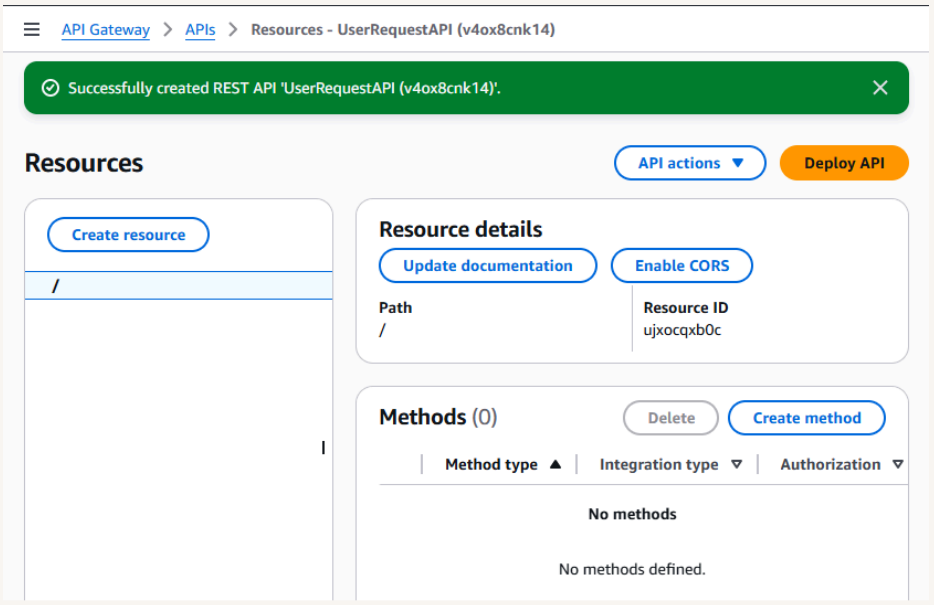


API Gateway

An API is a tool that allows different software systems to communicate and exchange data with each other. There are different types of APIs, such as REST APIs, HTTP APIs, and WebSocket APIs, each designed for different use cases. In this project, I am using a REST API, which communicates using standard HTTP methods like GET and works well with AWS Lambda and most programming languages.

Amazon API Gateway is an AWS service that allows developers to create, manage, secure, and monitor APIs. I'm using API Gateway in this project to create a public API endpoint that receives user requests, sends those requests to my AWS Lambda function for processing, and returns the Lambda response back to the user's browser.

When a user makes a request, such as clicking the "Get User Data" button, API Gateway receives and manages the incoming request. It acts as the front door for the application by handling traffic, checking authorization, and routing valid requests to the AWS Lambda function. Lambda then processes the request, retrieves the required data, and sends the response back through API Gateway to the user.

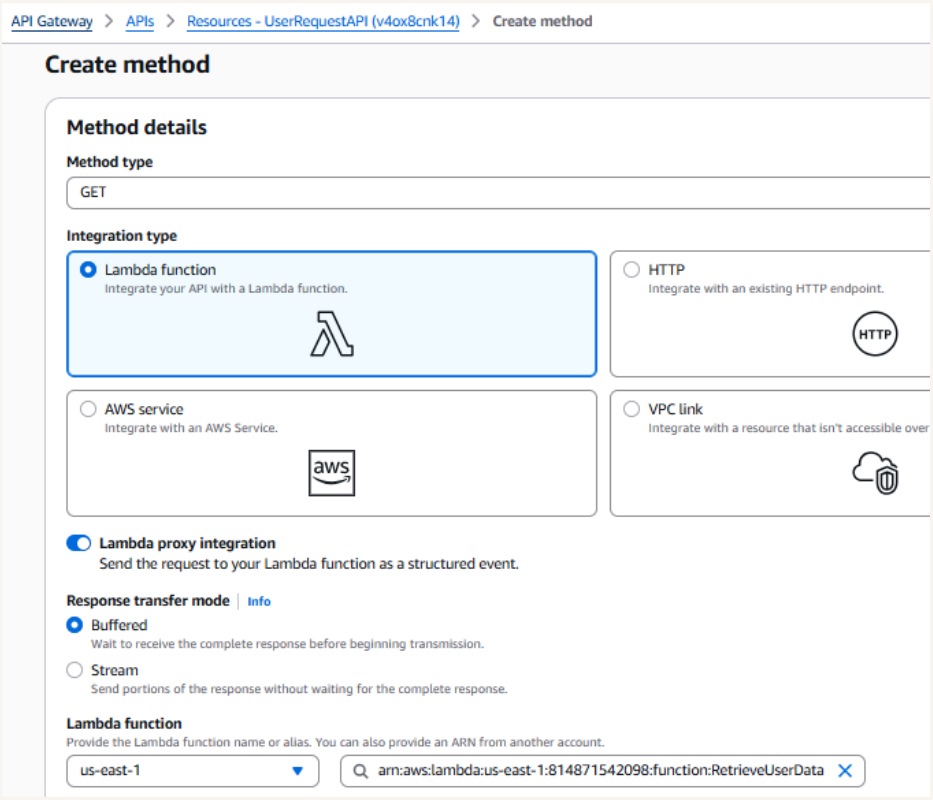

API Gateway > APIs > Resources - UserRequestAPI (v4ox8cnk14)
Successfully created REST API 'UserRequestAPI (v4ox8cnk14)'.
Resources
API actions
Deploy API
Create resource
/
Resource details
Update documentation
Enable CORS
Path
/
Resource ID
ujxocqxb0c
Methods (0)
Delete
Create method
Method type
Integration type
Authorization
No methods
No methods defined.

API Resources and Methods

An API is made up of resources, which are different sections or endpoints that organize the API's functionality. For example, an application's API could have separate resources for retrieving user data, product data, or messaging data, with each resource handling a different type of request within the same application.

Each API resource consists of methods, which define the specific actions that can be performed on that resource. In REST APIs, methods are based on standard HTTP methods such as GET to retrieve data, POST to create data, PUT to update data, and DELETE to remove data.

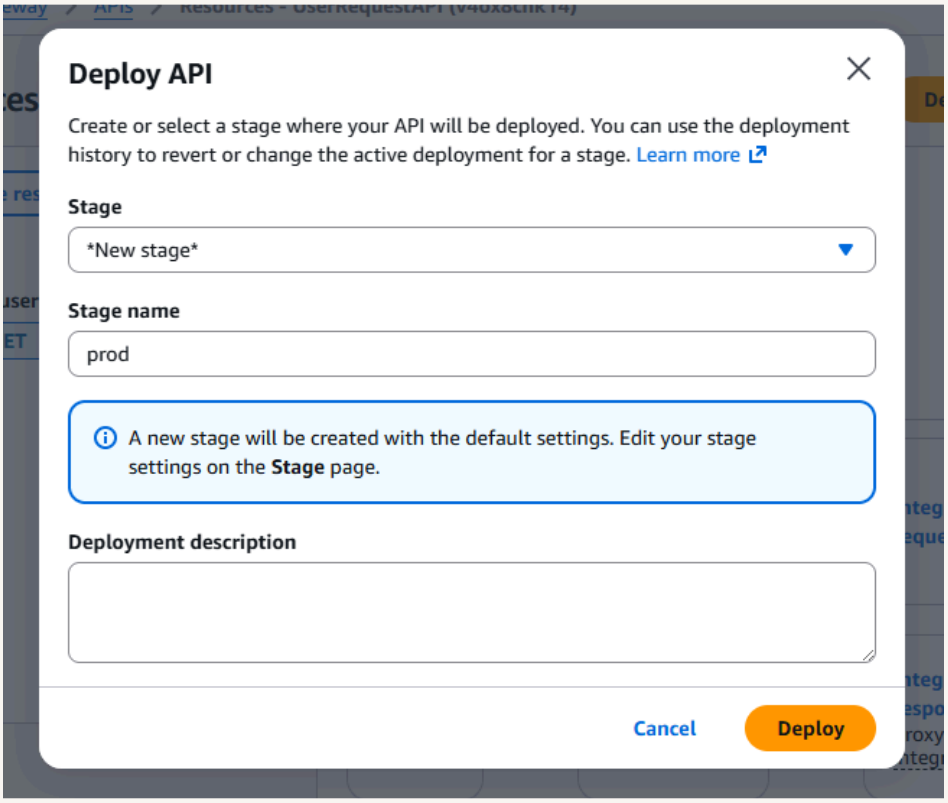
I created a GET method that connects my API Gateway resource with my AWS Lambda function. This means that when a user sends a request to the "/users" endpoint to retrieve data, API Gateway knows to forward that request to the Lambda function, which processes the request and returns the appropriate response.


API Gateway > APIs > Resources - UserRequestAPI (v4ox8cnk14) > Create method
Create method
Method details
Method type
GET
Integration type
Lambda function
Integrate your API with a Lambda function.
HTTP
Integrate with an existing HTTP endpoint.
HTTP
AWS service
Integrate with an AWS Service.
aws
VPC link
Integrate with a resource that isn't accessible over
Lambda proxy integration
Send the request to your Lambda function as a structured event.
Response transfer mode | Info
Buffered
Wait to receive the complete response before beginning transmission.
Stream
Send portions of the response without waiting for the complete response.
Lambda function
Provide the Lambda function name or alias. You can also provide an ARN from another account.
us-east-1
arn:aws:lambda:us-east-1:814871542098:function:RetrieveUserData

API Deployment

A stage is a deployed version or snapshot of an API that helps manage different environments and API versions. I deployed my API to the prod stage, which represents the production environment where live users and real application traffic can access and use the API.

To visit my API, I used the Invoke URL generated for the deployed prod stage in API Gateway. The API displayed an error because the Lambda function is trying to retrieve data from DynamoDB, but the DynamoDB table has not been created yet and the function does not yet have the required permissions to access it.



API Documentation

For my project's extension, I am writing API documentation because documentation helps other developers understand how to use the API correctly, including its endpoints, methods, and expected responses. Without clear documentation, it would be difficult for developers to integrate or interact with the API effectively. I can create and manage this documentation directly in the API Gateway console.

Once I prepared my documentation, I published it to the prod stage of my API in API Gateway. Documentation is published to a specific stage because different stages, such as development, testing, and production, can have different versions, configurations, or behaviors of the same API, so each stage may require its own documentation.

My downloaded documentation included both the manual documentation I wrote and additional automatically generated documentation created by API Gateway. The exported file also contained details about the API resources, methods, endpoints, and configurations, and it can be used with tools like Swagger or ReDoc to generate interactive and user-friendly API documentation webpages.


}
},
"x-amazon-apigateway-documentation" : {
"version" : "1",
"createdDate" : "2026-05-20T11:27:58Z",
"documentationParts" : [{
"location" : {
"type" : "API"
},
"properties" : {
"description" : "The UserRequestAPI manages user data requests and manipulation. It supports operations to get user
"baseURL" : "https://v4ox8cnk14.execute-api.us-east-1.amazonaws.com/"
}
}]
},
"x-amazon-apigateway-security-policy" : "TLS_1_0"
}

The place to learn & showcase your skills

Check out nextwork.org for more projects